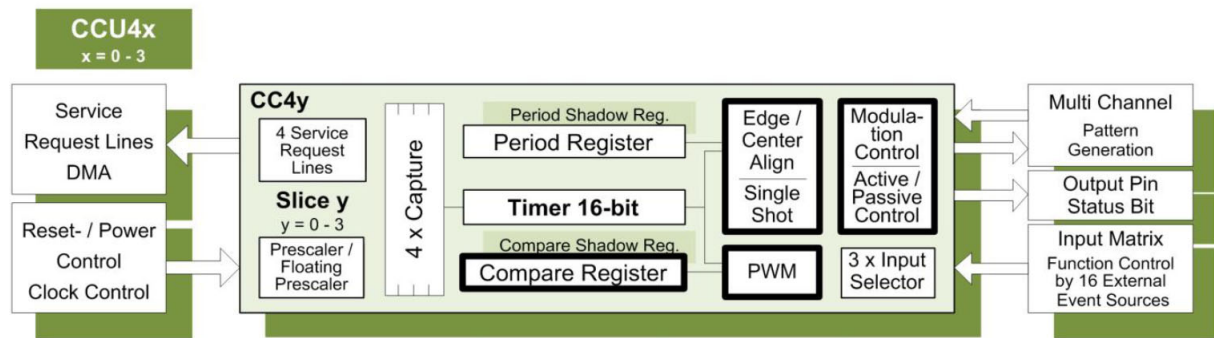


## XMC1200 Boot Kit – Capture/Compare Unit

### 1 Capture-Compare-Unit CCU4

Der Infineon-Mikrocontroller verfügt über mehrere Capture/Compare-Units (CCU), die unabhängig von der CPU mit Hilfe von Timer-Bausteinen zahlreiche Standardaufgaben übernehmen können. Dazu gehören z.B. die flexible Erzeugung von PWM-Signalen (Pulse-Width-Modulation), und das Monitoring von externen Signalen.

Eine CCU4-Einheit des XMC1200 Bausteins besteht dabei aus 4 Slices mit je einem 16-Bit-Zähler:

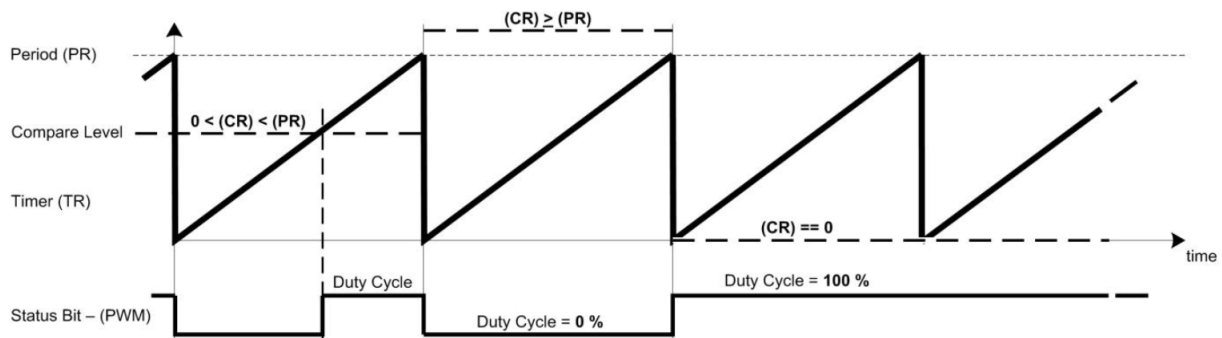


(DEV\_XMC1000\_XMC4000\_CCU4\_v1\_0, Figure 13)

Im normalen Betriebsmodus zählt der 16-Bit-Timer abhängig von Eingangsfrequenz  $f_{CCU4}$  aufwärts, bis der Wert im Period-Register (PR) erreicht ist. Der Zähler wird dann wieder auf Null gesetzt, und optional ein Interrupt ausgelöst. Erreicht der Zähler den Wert im Compare-Register (CR), kann ebenfalls ein Interrupt ausgelöst werden.

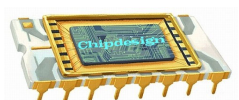
### 2 Erzeugung eines PWM-Signals

Die Register PR und CR können auf einfache Weise verwendet werden, um ein pulswidenmoduliertes Signal mit fast beliebiger Frequenz und beliebigem Tastverhältnis zu erzeugen:



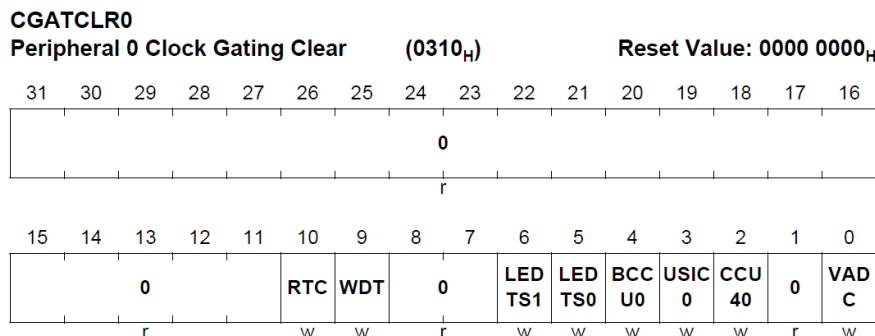
(DEV\_XMC1000\_XMC4000\_CCU4\_v1\_0, Figure 15)

Als Beispiel soll ein PWM-Signal mit 0,5 Hz und einem Duty-Cycle von 30% generiert werden, bei einem Prozessortakt von 32 MHz. Setzen Sie dazu ein neues Projekt mit dem Namen „Exercise\_5\_simplePWM“ auf. In main.c sind folgende Schritte notwendig:



## A. Taktsignal an den CCU4-Block anlegen (disable clock gating for CCU4)

Mit dem Register SCU->CGATCLR0 wird das Steuern (Gating) des Taktsignals von der Clock-System-Unit zu den verschiedenen Peripherie-Gruppen eingestellt:



| Field | Bits | Type | Description                                                                            |
|-------|------|------|----------------------------------------------------------------------------------------|
| VADC  | 0    | w    | VADC and SHS Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating |
| CCU40 | 2    | w    | CCU40 Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating        |
| USIC0 | 3    | w    | USIC0 Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating        |
| BCCU0 | 4    | w    | BCCU0 Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating        |

| Field  | Bits              | Type | Description                                                                      |
|--------|-------------------|------|----------------------------------------------------------------------------------|
| LEDTS0 | 5                 | w    | LEDTS0 Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating |
| LEDTS1 | 6                 | w    | LEDTS1 Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating |
| WDT    | 9                 | w    | WDT Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating    |
| RTC    | 10                | w    | RTC Gating Clear<br>0 <sub>B</sub> no effect<br>1 <sub>B</sub> disable gating    |
| 0      | 1, [8:7], [31:11] | r    | Reserved                                                                         |

Weshalb ist es sinnvoll, das Taktsignal für die einzelnen Peripherieelemente unabhängig schalten zu können?

Setzen Sie den Systemtakt auf 32 MHz, und schalten die Zuleitung der Clock  $f_{CCU4}$  zur CCU4-Einheit ein:

```
//-----CLOCK-SETUP-----
SCU_GENERAL->PASSWD = 0xC0; // Opens access to writing of all protected bits
SCU_CLK->CLKCR &= ~ 0xFFFF; // Clear IDIV and FDIV => 32 MHz
SCU_CLK->CGATCLR0 |= ____; // Disable gating for CCU40 clock
SCU_GENERAL->PASSWD = 0xC3; // Closes access for all protected bits
//-----
```

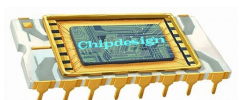
## B. Ausgang der CCU auf Pin P0.0 legen

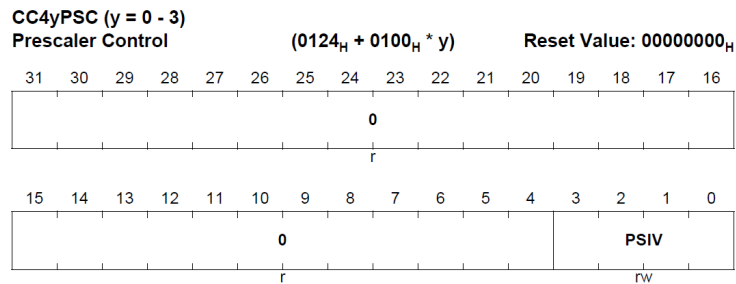
Für die Erzeugung des PWM-Signals soll die Unit 0 der CCU4-Einheit verwendet werden. Um das entsprechende Ausgangssignal CCU40.OUT0 auf den Pin P0.0 zu legen, muss im IO Control Register PORT0->IOCR0 die Funktion „Alternate output function 4“ (als push-pull) eingestellt werden (siehe dazu auch Übungsblatt ARM-1, und Table 21-13 im Reference-Manual):

```
// configure output of P0.0 to PWM signal (alternate function 4)
PORT0->IOCR0 = ____;
```

## C. Konfiguration des Prescalers

Innerhalb der CCU4-Einheit kann der Eingangstakt  $f_{CCU4}$  mit einem Prescaler auf die eigentliche Timer-Clock  $f_{tclk}$  heruntergeteilt werden. Dieser wird über dem Register CCU40\_CC40->PSC gesteuert und über dem Register CCU40->GIDLC eingeschaltet. Der Wert für das Prescal-Control-Register ergibt sich dabei folgendermaßen:





| Field | Bits   | Type | Description                                                                                                                                                                                                                                                                     |
|-------|--------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PSIV  | [3:0]  | rw   | <b>Prescaler Initial Value</b><br>This field contains the value that is applied to the Prescaler at startup.<br>When floating prescaler mode is used, this value is applied when a timer compare match AND prescaler compare match occurs or when a capture event is triggered. |
| 0     | [31:4] | r    | <b>Reserved</b><br>Read always returns 0.                                                                                                                                                                                                                                       |

| CC4yPSC.PSIV      | Resulting clock  |
|-------------------|------------------|
| 0000 <sub>B</sub> | $f_{ccu4}$       |
| 0001 <sub>B</sub> | $f_{ccu4}/2$     |
| 0010 <sub>B</sub> | $f_{ccu4}/4$     |
| 0011 <sub>B</sub> | $f_{ccu4}/8$     |
| 0100 <sub>B</sub> | $f_{ccu4}/16$    |
| 0101 <sub>B</sub> | $f_{ccu4}/32$    |
| 0110 <sub>B</sub> | $f_{ccu4}/64$    |
| 0111 <sub>B</sub> | $f_{ccu4}/128$   |
| 1000 <sub>B</sub> | $f_{ccu4}/256$   |
| 1001 <sub>B</sub> | $f_{ccu4}/512$   |
| 1010 <sub>B</sub> | $f_{ccu4}/1024$  |
| 1011 <sub>B</sub> | $f_{ccu4}/2048$  |
| 1100 <sub>B</sub> | $f_{ccu4}/4096$  |
| 1101 <sub>B</sub> | $f_{ccu4}/8192$  |
| 1110 <sub>B</sub> | $f_{ccu4}/16384$ |
| 1111 <sub>B</sub> | $f_{ccu4}/32768$ |

Setzen sie den Prescaler derart, dass die Eingangsfrequenz auf  $f_{tclk} \approx 8$  kHz heruntergeteilt wird. Bitte beachten Sie, dass die Frequenz an der Peripherie  $f_{PCLK} = f_{CCU4}$  generell doppelt so hoch ist als der Prozessortakt  $f_{MLCK}$  (siehe auch Blockschaltbild der Takterzeugung auf der ersten Seite von Übungsblatt ARM3):

```
CCU40->CC40->PSC = ____; // initial prescaler value, corresponds to ~7.8 kHz
CCU40->GIDLC      |= 0x100; // start prescaler
```

## D. Setzen der Periode und des Taktverhältnisses

Die Werte für das Period- und Compare-Register werden zuerst in die entsprechenden Shadow-Register PRS und CRS geladen. Erst, wenn der obere Zählerstand erreicht ist, überträgt die Hardware den Inhalt der Shadow-Register PRS und CRS in die eigentlichen Register PR und CR. Dadurch wird sichergestellt, dass eine Änderung synchron übernommen wird, und somit keine kurzen „Glitches“ entstehen. Der Transfer von PRS/CRS nach PR/CR muss im Register GCSS eingeschaltet werden.

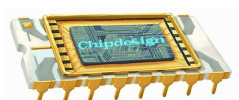
Die Frequenz und das Tastverhältnis des Ausgangssignals lassen sich dann folgendermaßen bestimmen (siehe auch Abbildung auf Seite 1):

$$f_{PWM} = f_{tclk} / (PR + 1)$$

$$D_{PWM} = 1 - CR / (PR + 1)$$

Bestimmen Sie die Werte für PRS und CRS derart, dass ein 0,5 Hz Signal mit einem Duty-Cycle von 30% erzeugt wird:

```
CCU40->CC40->PRS = ____; // period register
CCU40->CC40->CRS = ____; // compare register
CCU40->GCSS      |= 0x01; // transfer shadow registers to PR and CR
```



## E. Starten des Timers

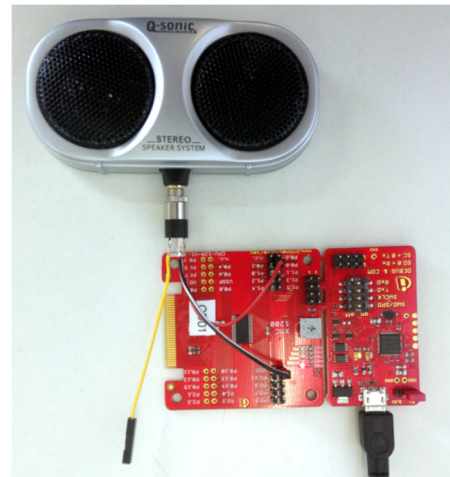
Schließlich muss der Timer noch über die Register GIDLC und TCSET geestartet werden:

```
CCU40->GIDLC   |= 0x01;  // remove CCU4_0 slice from idle mode
CCU40->CC40->TCSET |= 0x01;  // start timer
```

Vervollständigen Sie das Programm, und testen Sie es auf dem Evaluierungsboard. Die LED P0.0 sollte mit 0,5 Hz blinken. Stimmt das Tastverhältnis?

## 3 Erzeugung eines 2 kHz Signals

Schreiben Sie eine C-Funktion namens setFrequency, die als Übergabeparameter eine Frequenz in Hz erhält, und ein entsprechendes PWM-Signal auf P0.0 ausgibt. Der Duty-Cycle soll dabei ca. 12.5% sein (1/8). Den Prescaler stellen Sie so ein, dass die Eingangsfrequenz in die CCU4-Unit  $f_{tclk} = 125 \text{ kHz}$  beträgt. Erzeugen Sie ein 2 kHz-Signal, **testen es mit Hilfe eines Oszilloskops** und mit einem Lautsprecher:



## 4 Spielen der Tonleiter

Erstellen Sie ein neues Projekt mit einem Programm, das die 8 Töne der C-Dur-Tonleiter spielt. Jeder Ton soll dabei für etwa 0,5 s gehalten werden. Die Frequenzen der Töne ergeben sich aus folgender Tabelle:

| Ton | Frequenz/Hz |
|-----|-------------|
| C   | 262         |
| D   | 294         |
| E   | 330         |
| F   | 349         |
| G   | 392         |
| A   | 440         |
| H   | 494         |
| C   | 523         |

Verwenden Sie zur Umstellung der Tonhöhe die Interrupt-Funktion des Prozessors entsprechend dem vorherigen Übungsblatt. Zeigen Sie Ihr Ergebnis dem Betreuer.

